

DevOps Handover: AWS CloudFront Video Streaming Setup

Target Audience: DevOps / SRE / Deployment Team

System: TeamManagerProduction API & Frontend

Feature: Secure AWS CloudFront Edge Video Streaming

Overview

This guide contains the exact steps required by the deployment team to provision AWS CloudFront for secure video streaming, configure production environment variables, and verify deployment.

[!IMPORTANT]

No S3 Bucket Policy Changes Needed:

Existing S3 bucket policies and permissions remain **100% untouched**. CloudFront acts as an edge delivery accelerator in front of the bucket.

Quick Reference: What You Need from AWS

At the end of this setup, you will supply these 4 values to the production environment:

1. `CLOUDFRONT_DOMAIN` (e.g. `d13xjfi0ctxoet.cloudfront.net`)
 2. `CLOUDFRONT_KEY_ID` (e.g. `K3ABC123EXAMPLE`)
 3. `CLOUDFRONT_PRIVATE_KEY_BASE64` (One-line base64 string of the private RSA key)
 4. `ENABLE_CLOUDFRONT_STREAMING=True`
-

Step 1: Generate RSA Key Pair

CloudFront requires a 2048-bit RSA key pair to sign and verify secure media URLs.

Run these commands in a secure terminal:

```
# 1. Generate the RSA private key
openssl genrsa -out cloudfront_private_key.pem 2048

# 2. Extract the public key in standard PEM format
openssl rsa -pubout -in cloudfront_private_key.pem -out cloudfront_public_key.pem

# 3. Generate a one-line Base64 string for the private key (for environment variables)
```

```
base64 -w 0 cloudfront_private_key.pem > cloudfront_private_key.base64.txt
```

[!CAUTION]

Store `cloudfront_private_key.pem` and `cloudfront_private_key.base64.txt` in your secure secrets manager (AWS Secrets Manager, 1Password, or Vault). **Never commit private keys to Git.**

Step 2: Configure CloudFront Key Management in AWS

1. Log in to the [AWS Management Console](#) and go to **CloudFront**.
2. In the left sidebar, expand **Key management**:

2.1 Create Public Key

- Click **Public keys** -> **Create public key**.
- **Name:** `teammanager-prod-video-key`
- **Key value:** Paste the entire content of `cloudfront_public_key.pem` (including `-----BEGIN PUBLIC KEY-----` and `-----END PUBLIC KEY-----`).
- Click **Create public key**.
- **Copy the generated Key ID** (e.g. `K3ABC123EXAMPLE`). You will need this for `CLOUDFRONT_KEY_ID`.

2.2 Create Key Group

- In the left sidebar, click **Key groups** -> **Create key group**.
- **Name:** `teammanager-prod-video-key-group`
- **Public keys:** Check `teammanager-prod-video-key`.
- Click **Create key group**.

Step 3: Create the CloudFront Distribution

1. In the CloudFront console, click **Distributions** -> **Create distribution**.

Origin Settings

Field	Value	Notes
Origin domain	Select your production S3 bucket	e.g. <code>.com</code>
Origin path	<i>Leave blank</i>	Django automatically appends the <code>media/</code> prefix

Name	Auto-filled	Keep default
Origin access	Public (or existing access setup)	Bucket policies do not need modification

Default Cache Behavior

Field	Value	Notes
Viewer protocol policy	Redirect HTTP to HTTPS	Enforces secure streaming
Allowed HTTP methods	GET, HEAD, OPTIONS	Required for video and CORS preflight
Restrict viewer access	Yes	Secures videos behind signed URLs
Trusted authorization type	Trusted key groups (recommended)	
Add key groups	Select roup	Created in Step 2.2
Cache policy	Managed-CachingOptimized	Critical: Caches HTTP 206 Range headers for smooth scrubbing
Origin request policy	Managed-CORS-S3origin	Passes CORS headers to S3
Response headers policy	dersPolicy	Adds required CORS response headers

Distribution Settings

Field	Value	Notes
Price class	Select based on target region	e.g. <i>Use North America, Europe, Asia, Middle East, and Africa</i>
Alternate domain name (CNAME)	<i>Optional</i>	e.g. cdn-video.teammanager.ng (requires ACM certificate) or leave blank
Description	Streaming CDN	

2. Click **Create distribution**.
3. Wait 3–5 minutes for the status to show **Enabled**.
4. Copy your **Distribution domain name** (e.g. **d13xfi0ctxoet.cloudfront.net**).

Step 4: Add Environment Variables in Production

Add these variables to the backend environment (`.env`, Docker Compose, AWS ECS task definition, or Kubernetes Secret):

```
# ?? AWS CloudFront Video Streaming ?????????????????????????????????????????????????????????????
ENABLE_CLOUDFRONT_STREAMING=True
CLOUDFRONT_DOMAIN=d13xjfi0ctxoet.cloudfront.net
CLOUDFRONT_KEY_ID=K3ABC123EXAMPLE
CLOUDFRONT_MEDIA_PREFIX=media
CLOUDFRONT_URL_EXPIRATION_SECONDS=7200

# Paste the full one-line Base64 string from cloudfront_private_key.base64.txt:
CLOUDFRONT_PRIVATE_KEY_BASE64=LS0tLS1CRUdJTiBSU0EgUFJJVkFURSBLRVktLS0tLQ...
```

[!TIP]

Why Base64? Using `CLOUDFRONT_PRIVATE_KEY_BASE64` allows containerized deployments without mounting file volumes or baking private keys into Docker images.

Step 5: Deploy & Restart Backend Services

Restart the backend API workers to load the new environment variables:

```
# Docker Compose example:
docker compose -f docker-compose.prod.yml restart web worker

# Kubernetes example:
kubectl rollout restart deployment/teammanager-backend
```

Step 6: Post-Deployment Verification

1. Fast CLI Test (Run inside backend container)

Verify key loading and URL generation:

```
python manage.py shell -c "
from digital_products.cloudfront_signer import generate_signed_video_url
url = generate_signed_video_url('digital_products/courses/test.mp4')
print('Generated Signed URL successfully:')
print(url)
assert 'Signature=' in url
assert 'Key-Pair-Id=' in url
print('OK: CloudFront signing is functional!')
```

"

2. Verify CloudFront Edge Video Delivery (cURL)

Take the URL generated above and test byte-range delivery directly from CloudFront:

```
curl -I -H "Range: bytes=0-1024" "<signed-url>"
```

Expected Response:

- Status: `HTTP/2 206 Partial Content` (or `HTTP/1.1 206 Partial Content`)
- Header: `content-range: bytes 0-1024/...`
- Header: `x-cache: Hit from cloudfront` (or `Miss from cloudfront` on first hit)

3. Student Playback Check (Browser)

1. Log in as an enrolled student and navigate to a purchased course.
2. Open **Developer Tools** -> **Network** tab.
3. Observe `GET /api/.../stream-url/` returning `{"url": "https://d13...cloudfront.net/..."}` with status `200 OK`.
4. Observe the `<video>` element requesting byte ranges directly from `*.cloudfront.net` with status `206 Partial Content`.
5. Scrub forward and backward on the video player: Seeking should be instant with zero stutter.

Step 7: Emergency Rollback Procedure

If CloudFront experiences an outage, invalid key configuration, or AWS misconfiguration, you can revert instantly to direct S3 streaming with **zero downtime**:

1. In production `.env`, change:

```
ENABLE_CLOUDFRONT_STREAMING=False
```

2. Restart backend services:

```
docker compose -f docker-compose.prod.yml restart web worker
```

3. The API and video player will immediately fall back to standard S3 streaming. No database migrations, frontend deployments, or downtime required.
-

Troubleshooting Guide

Issue	Root Cause	Solution
CloudFront returns AccessDenied XML	Key ID mismatch or key expired.	Verify CLOUDFRONT_KEY_ID matches the Public Key ID in CloudFront. Ensure server clock is in sync via NTP.
ivate key is not configured	Missing environment variable.	Ensure E64 is populated and has no trailing whitespaces or line breaks.
CORS error in browser console	Origin Request Policy or Response Headers Policy missing.	In CloudFront Behavior, set Origin request policy to Managed-CORS-S3Origin and Response headers policy to eadersPolicy .
Video fails to seek or scrub	Cache policy stripping Range headers.	In CloudFront Behavior, set Cache Policy to Managed-CachingOptimized .